PhpSaga

Agentic Implementation Prompts

6 Versions · Step-by-Step · Paste & Execute

v1.0 PHP Core | v1.5 Laravel | v2.0 Symfony | v2.5 Persist | v3.0 Node.js | v4.0 Python

2026 · Draft v0.1 · For Claude Agent

How to Use This Document

■ WORKFLOW — Follow in this exact order

1. Open the chapter for the version you are implementing
2. Read the CONTEXT block at the top (role, goal, constraints)
3. Copy the full SYSTEM PROMPT and paste it as the agent system prompt
4. Execute each STEP CARD in order — confirm completion before next step
5. Verify every item in the EXPECTED OUTCOMES checklist
6. Only then move to the next version

■ CRITICAL RULES — Never violate these

NEVER skip a step — each step builds on the previous one

NEVER start a new version before the current one passes ALL tests

NEVER merge steps — one step = one focused implementation unit

ALWAYS confirm: 'Step N complete and tested' before sending Step N+1

ALWAYS include the full SYSTEM PROMPT at the start of every new conversation

If the agent produces incorrect output — stop, debug, fix before continuing

■ BUILD ORDER — Dependencies matter

v1.0 PHP Core → Foundation. Must be first. Everything depends on this.

v1.5 Laravel → Requires v1.0 complete.

v2.0 Symfony → Requires v1.0 complete. Can run parallel with v1.5.

v2.5 Persistence → Requires v1.0 complete. Build after v1.5 + v2.0.

v3.0 Node.js → Independent TypeScript rewrite. Requires v1.0 as reference.

v4.0 Python → Independent Python rewrite. Requires v1.0 as reference.

v1.0

Core PHP Transaction Library

The foundation of PhpSaga. Pure PHP 8.1+. Zero external dependencies.

PHP 8.1+ · Composer · PHPUnit 10 · Strict Types · PSR-4

■ AGENT CONTEXT — Read before starting

ROLE: You are a senior PHP engineer with deep knowledge of SOLID principles,
design patterns, and production-grade library architecture.
GOAL: Build PhpSaga v1.0 — a PHP library that wraps any set of functions
in a transactional execution block with automatic compensation.
OUTPUT: A fully working, installable Composer package with passing PHPUnit tests.
QUALITY: Production-grade code. No shortcuts. No TODOs in final output.
TESTS: Every step must be tested before moving to the next step.

■ ABSOLUTE CONSTRAINTS

PHP 8.1+ ONLY — use enums, readonly properties, named args, match expressions
declare(strict_types=1) at the top of EVERY file — no exceptions
NO external dependencies except PHPUnit in require-dev
ALL public methods must have complete PHPDoc blocks with @param, @return, @throws
ALL exceptions must carry descriptive messages with context arrays
CompensationEngine MUST run rollbacks in REVERSE order — this is not optional
CompensationEngine MUST only compensate COMPLETED steps — never failed or pending
Step::make() MUST throw if a side-effect step has no ->rollback() defined

IMPLEMENTATION STEPS — Execute in strict order

1 composer.json — Package manifest

```
{
"name": "phpsaga/phpsaga",
"description": "Transactional execution for any PHP function",
"type": "library",
"require": { "php": "^8.1" },
"require-dev": { "phpunit/phpunit": "^10.0" },
"autoload": {
```

```
"psr-4": { "PhpSaga\\": "src/" }
},
"autoload-dev": {
"psr-4": { "PhpSaga\\Tests\\": "tests/" }
},
"minimum-stability": "stable"
}
```

2 src/Step.php — Fluent step builder

```
<?php declare(strict_types=1);
namespace PhpSaga;

final class Step {
// Rollback failure strategy constants
public const RETRY = 1;
public const LOG_AND_CONTINUE = 2;
public const ALERT_HUMAN = 3;
public const THROW = 4;

private function __construct(
private readonly string $name,
private readonly mixed $action,
private mixed $compensation = null,
private int $rollbackStrategy = self::LOG_AND_CONTINUE,
) {}

public static function make(callable $action, string $name = ''): self { ... }
public function rollback(callable $compensation): self { ... }
public function onRollbackFailure(int $strategy): self { ... }

// ENFORCE: throw if action is set but compensation is null
// Message: "Step '{name}' produces side effects. You MUST define ->rollback()"
public function validate(): void { ... }

// Getters: getName(), getAction(), getCompensation(), getRollbackStrategy()
}
```

3 src/Engine/StateTracker.php — Real-time step state

```
<?php declare(strict_types=1);
```

```
namespace PhpSaga\Engine;

final class StateTracker {
// Internal: array<string, array{status:string, result:mixed, error:?Throwable}>

public function markStarted(string $name): void;
public function markCompleted(string $name, mixed $result = null): void;
public function markFailed(string $name, \Throwable $e): void;

// Returns steps in COMPLETION ORDER — used by CompensationEngine
public function getCompletedSteps(): array;

public function wasStepCompleted(string $name): bool;
public function getFailedStep(): ?array;
public function reset(): void;
}
```

4 src/Log/AuditLogger.php — Immutable event log

```
Events to log (use these exact string constants):
STEP_STARTED | STEP_COMPLETED | STEP_FAILED
COMPENSATION_STARTED | COMPENSATION_COMPLETED | COMPENSATION_FAILED
TRANSACTION_COMPLETED | TRANSACTION_ROLLED_BACK

public function log(string $event, array $context = []): void;
public function getLogs(): array; // append-only, never modifiable

Each log entry must contain:
[ 'event' => string, 'context' => array, 'timestamp' => float (microtime) ]
```

5 src/Engine/ExecutionEngine.php — Sequential runner

```
__construct(StateTracker $tracker, AuditLogger $logger)

execute(array $steps): void
foreach $steps as $step:
1. $step->validate() — throws if rollback missing
2. $logger->log(STEP_STARTED, ['step' => $step->getName()])
3. $tracker->markStarted($step->getName())
4. try: call $step->getAction()()
5. $tracker->markCompleted($name, $result)
```

```
6. $logger->log(STEP_COMPLETED, [...])
7. catch(Throwable $e):
8. $tracker->markFailed($name, $e)
9. $logger->log(STEP_FAILED, ['error' => $e->getMessage()])
10. throw new TransactionFailedException($name, $e)

TransactionFailedException must carry: stepName, originalException, context[]
```

6 src/Failure/RollbackFailHandler.php — Compensation failure handler

```
__construct(AuditLogger $logger)
registerAlertCallback(callable $callback): void

handle(Step $step, Throwable $e): void
switch $step->getRollbackStrategy():

RETRY:
Attempt compensation up to 3 times
Delays: 100ms → 200ms → 400ms (exponential backoff)
If all 3 fail: log COMPENSATION_FAILED, continue

LOG_AND_CONTINUE:
Log COMPENSATION_FAILED with error context
Do NOT throw — remaining compensations must still fire

ALERT_HUMAN:
Log at CRITICAL level: ['step'=>$name, 'error'=>$msg, 'severity'=>'CRITICAL']
Fire registered alert callback if set
Continue — do not halt

THROW:
Re-throw $e immediately — halts all further compensation
```

7 src/Engine/CompensationEngine.php — Reverse rollback orchestrator

```
__construct(StateTracker $tracker, RollbackFailHandler $handler, AuditLogger $logger)

compensate(array $steps): void

ALGORITHM (this is the core of PhpSaga — implement exactly):
1. Get $completedSteps = $tracker->getCompletedSteps() // in completion order
```

```
2. $toCompensate = array_reverse($completedSteps) // REVERSE order
3. For each $completedStep:
a. Find matching Step object from $steps by name
b. If $step->getCompensation() is null → skip (pure DB op, auto-handled)
c. $logger->log(COMPENSATION_STARTED, ['step' => $name])
d. try: call $step->getCompensation()()
e. $logger->log(COMPENSATION_COMPLETED, [...])
f. catch(Throwable $e):
g. $logger->log(COMPENSATION_FAILED, [...])
h. $handler->handle($step, $e) // delegates strategy

CRITICAL: Failed step is NOT in $completedSteps — never compensated
CRITICAL: Steps that never ran are NOT in $completedSteps — never compensated
```

8 src/Transaction.php — Public entry point

```
static run(array $steps, ?PDO $db = null): TransactionResult

EXECUTION FLOW:
1. Instantiate: StateTracker, AuditLogger, ExecutionEngine, CompensationEngine
2. If $db: $db->beginTransaction()
3. try:
ExecutionEngine->execute($steps)
If $db: $db->commit()
$logger->log(TRANSACTION_COMPLETED)
return TransactionResult::success($logger->getLogs())
4. catch(TransactionFailedException $e):
CompensationEngine->compensate($steps)
If $db: $db->rollBack()
$logger->log(TRANSACTION_ROLLED_BACK)
return TransactionResult::failure($e, $logger->getLogs())

TransactionResult must expose:
wasSuccessful(): bool
getAuditLog(): array
getFailedStep(): ?string
getException(): ?Throwable
```

9 tests/ — PHPUnit test suite (all must pass)

```
Test 1: Happy path
```

```
All 4 steps complete → result->wasSuccessful() === true
DB PDO commit() called exactly once
Audit log contains TRANSACTION_COMPLETED

Test 2: Failure on step 2 of 4
Step 1 compensated ✓ | Step 2 NOT compensated ✓
Steps 3 + 4 never ran → NOT compensated ✓
DB PDO rollBack() called exactly once ✓

Test 3: Compensation order verification
4 steps all complete → step 4 fails compensation trigger
Compensation call order logged: [4, 3, 2, 1] (strict reverse) ✓

Test 4: RETRY strategy
Compensation callable throws 3 times → handler retried 3 times
4th call (LOG_AND_CONTINUE fallback) → does not throw ✓

Test 5: LOG_AND_CONTINUE
Step 2 compensation fails → step 1 compensation still fires ✓
Both failures logged in audit log ✓

Test 6: Step without ->rollback() throws
Step::make(fn()=>'side-effect') without ->rollback()
Transaction::run() throws with message containing 'side effects' ✓
```

10 README.md — Documentation

```
Required sections:
1. One-line description
2. Installation: composer require phpsaga/phpsaga
3. Quick start — the full OTP scenario with compensation
4. The 3 operation types (Reversible / Compensatable / Irreversible)
5. API reference table (all public methods)
6. Rollback failure strategies (all 4 with examples)
7. Honest limitations section
8. 'Why not just try/catch?' explanation
9. Contributing guide
```

■ EXPECTED OUTCOMES — VERIFY BEFORE MARKING DONE

- ✓ composer install completes with zero errors
- ✓ All 6 PHPUnit tests pass — zero failures, zero skipped
- ✓ Step without ->rollback() throws exception containing phrase 'side effects'
- ✓ Compensation runs in exact reverse order — verified by Test 3
- ✓ Failed step does NOT appear in compensation log
- ✓ Steps that never ran do NOT appear in compensation log
- ✓ PDO commit() called on success, rollBack() called on failure
- ✓ RETRY strategy attempts exactly 3 times with increasing delays
- ✓ LOG_AND_CONTINUE — subsequent compensations still fire after a failure
- ✓ AuditLogger contains complete chronological event timeline
- ✓ README renders on GitHub with all code examples valid PHP

v1.5

Laravel Package

First-class Laravel DX. Facade, ServiceProvider, Artisan, Middleware.

PHP 8.1+ · Laravel 10/11 · Composer · PHPUnit · Pest (optional)

■ **PREREQUISITE: PhpSaga v1.0 is complete. All 6 tests pass. Package installable via Composer.**

■ AGENT CONTEXT

ROLE: Senior Laravel package developer familiar with service containers,

facades, event systems, and Laravel conventions.

GOAL: Wrap PhpSaga v1.0 in a Laravel package that feels 100% native.

Developers should not need to know about the core library directly.

OUTPUT: An auto-discoverable Laravel package with Facade, Artisan command,

HTTP middleware, and DB auto-integration.

QUALITY: Follow all Laravel conventions. Zero breaking changes to v1.0 core.

■ ABSOLUTE CONSTRAINTS

MUST support both Laravel 10 and Laravel 11 — test on both

MUST use auto-discovery — zero manual registration required from users

MUST NOT modify phpsaga/phpsaga core in any way

Facade MUST proxy all methods from Transaction:: and Step:: transparently

Middleware MUST register as named alias 'transactional' — not by class name

SagaStep interface MUST be accepted by Transaction::run() alongside Step objects

Config file MUST have inline comments explaining every key

All default config values MUST be safe for production use

IMPLEMENTATION STEPS — Execute in strict order

1 composer.json — Laravel package manifest

```
{
"name": "phpsaga/laravel",
"require": {
"php": "^8.1",
```

```
"phpsaga/phpsaga": "^1.0",
"laravel/framework": "^10.0|^11.0"
},
"extra": {
"laravel": {
"providers": ["PhpSaga\\Laravel\\PhpSagaServiceProvider"],
"aliases": { "Saga": "PhpSaga\\Laravel\\Facades\\Saga" }
}
},
"autoload": { "psr-4": { "PhpSaga\\Laravel\\": "src/" } }
}
```

2 config/phpsaga.php — Configuration file

```
return [
// Whether to persist audit log to the database
// Requires running: php artisan migrate
'log_to_database' => false,

// Laravel log channel for ALERT_HUMAN strategy
// Must match a channel defined in config/logging.php
'alert_channel' => 'stack',

// How many times to retry a failed compensation (RETRY strategy)
'retry_attempts' => 3,

// Initial delay in milliseconds between retry attempts (doubles each attempt)
'retry_delay_ms' => 100,
];
```

3 src/PhpSagaServiceProvider.php — Auto-discovered provider

```
class PhpSagaServiceProvider extends ServiceProvider {

public function register(): void {
$this->mergeConfigFrom(__DIR__.'/../config/phpsaga.php', 'phpsaga');
$this->app->singleton('phpsaga', function($app) {
// Wire config values into Transaction and RollbackFailHandler
// Return configured Transaction factory/runner
});
}
```

```
public function boot(): void {
$this->publishes([
__DIR__.'/../config/phpsaga.php' => config_path('phpsaga.php'),
], 'phpsaga-config');
$this->publishes([
__DIR__.'/../database/migrations' => database_path('migrations'),
], 'phpsaga-migrations');
$this->commands([MakeSagaCommand::class]);
$this->app['router']->aliasMiddleware('transactional', TransactionalRequest::class);
}
}
```

4 src/Facades/Saga.php — Static facade

```
class Saga extends Facade {
protected static function getFacadeAccessor(): string {
return 'phpsaga';
}
}

// Facade must expose these methods with correct docblocks:
// @method static TransactionResult run(array $steps, ?string $connection = null)
// @method static Step step(callable $action, string $name = '')

// Usage example for README:
// Saga::run([
// Saga::step(fn() => sendSMS($user))->rollback(fn() => cancelSMS($user)),
// Saga::step(fn() => storeInDB($user)),
// ]);
```

5 src/Contracts/SagaStep.php — Class-based step interface

```
interface SagaStep {
/**
* The main action to execute.
* @return mixed Result of the action (passed to audit log)
*/
public function execute(): mixed;

/**
```

```
 * Compensation action — called if a later step fails.
 * MUST be idempotent — safe to call multiple times.
 */
public function compensate(): void;

/**
 * Failure strategy for this step's compensation.
 * @return int One of Step::RETRY|LOG_AND_CONTINUE|ALERT_HUMAN|THROW
 */
public function onCompensationFailure(): int;
}

// Transaction::run() MUST detect SagaStep instances and wrap them
// in Step objects automatically — callers never use Step directly
```

6 src/Console/MakeSagaCommand.php — Artisan generator

```
// Command: php artisan make:saga {name}
// Example: php artisan make:saga ProcessCheckout
// Output: app/Sagas/ProcessCheckoutSaga.php

// Generated stub must contain:
// - Class ProcessCheckoutSaga
// - Use statement: use PhpSaga\Laravel\Facades\Saga;
// - Method: public function execute(): TransactionResult
// - Saga::run([]) call with 3 commented-out Step examples
// - Each example step has a ->rollback() defined
// - PHPDoc block explaining how to use the saga
// - Example of how to call from a controller
```

7 src/Http/Middleware/TransactionalRequest.php — Route wrapper

```
class TransactionalRequest {
public function handle(Request $request, Closure $next): Response {
DB::beginTransaction();
try {
$response = $next($request);
DB::commit();
return $response;
} catch (\Throwable $e) {
DB::rollBack();
```

```
throw $e;
}
}
}

// Register as named alias 'transactional' in ServiceProvider
// Usage: Route::post('/checkout', ...)->middleware('transactional')
// Note: This wraps the DB transaction — PhpSaga compensation fires
// via the controller's own Saga::run() calls inside the route
```

8 database/migrations/..._create_saga_audit_log_table.php

```
Schema::create('phpsaga_audit_log', function (Blueprint $table) {
$table->id();
$table->uuid('transaction_id')->index();
$table->string('step_name')->nullable();
$table->string('event', 50);
$table->json('context')->nullable();
$table->timestamps();
$table->index('event');
});

// This migration only runs when config('phpsaga.log_to_database') is true
// Guard the migration with a config check
```

9 tests/ — Feature + Unit tests

```
Feature Test 1: Facade resolves
Saga::run([...steps...]) resolves without error ✓
Returns TransactionResult instance ✓

Feature Test 2: SagaStep interface
Create concrete SagaStep class
Pass instance to Saga::run()
execute() and compensate() called correctly ✓

Feature Test 3: Artisan command
php artisan make:saga TestSaga
File exists at app/Sagas/TestSaga.php ✓
File contains 'Saga::run' ✓
File contains '->rollback(' ✓
```

```
Feature Test 4: Transactional middleware
Route with 'transactional' middleware
Exception in route handler → DB::rollBack() called ✓
Success in route handler → DB::commit() called ✓

Feature Test 5: Config override
Set config('phpsaga.retry_attempts', 5)
Verify retry attempts = 5, not 3 ✓

Both Laravel 10 and Laravel 11 must pass all tests.
```

■ EXPECTED OUTCOMES — VERIFY BEFORE MARKING DONE

- ✓ Package auto-discovered — zero manual registration in config/app.php
- ✓ Saga::run() and Saga::step() work with correct return types
- ✓ SagaStep interface instances accepted alongside Step objects in run()
- ✓ php artisan make:saga generates correct stub with rollback examples
- ✓ ->middleware('transactional') wraps route in DB::beginTransaction/commit
- ✓ php artisan vendor:publish --tag=phpsaga-config copies config correctly
- ✓ All 5 feature tests pass on Laravel 10 AND Laravel 11
- ✓ Config file has inline comments on every key

v2.0

Symfony Bundle

PHP 8 Attributes, autowiring, Doctrine EntityManager integration.

PHP 8.1+ · Symfony 6/7 · Doctrine ORM · Monolog · DI Container

■ PREREQUISITE: PhpSaga v1.0 is complete. All 6 tests pass. Package installable via Composer.

■ AGENT CONTEXT

ROLE: Senior Symfony bundle developer. Expert in DependencyInjection,

EventDispatcher, Doctrine ORM, and Symfony bundle conventions.

GOAL: Build a Symfony Bundle so developers can annotate controller methods

with #[Transactional] and get automatic saga execution.

OUTPUT: Installable Symfony bundle. Auto-wired SagaRunner service.

#[Transactional] attribute works on any controller method.

QUALITY: Follow Symfony bundle best practices. Constructor injection everywhere.

No service locators. No static calls.

■ ABSOLUTE CONSTRAINTS

MUST support Symfony 6.x AND Symfony 7.x — test on both

MUST use constructor injection everywhere — NO container->get() calls

MUST NOT use static methods anywhere in the bundle (except the Attribute class)

#[Transactional] MUST detect on controller methods via PHP Reflection

SagaRunner MUST be injectable — tagged as a public service in services.yaml

SagaStepInterface implementations MUST auto-discover via DI tagging (phpsaga.step)

Doctrine EntityManager MUST rollback on any Throwable — not just exceptions

Bundle configuration MUST validate via Symfony Config component

IMPLEMENTATION STEPS — Execute in strict order

1 PhpSagaBundle.php + composer.json

```
// PhpSagaBundle.php
namespace PhpSaga\Symfony;
use Symfony\Component\HttpKernel\Bundle\AbstractBundle;
```

```
class PhpSagaBundle extends AbstractBundle {
public function getPath(): string { return __DIR__; }
}

// composer.json
// name: "phpsaga/symfony"
// require: phpsaga/phpsaga ^1.0, symfony/framework-bundle ^6.0|^7.0
// extra.symfony.require: "^6.0|^7.0"
// autoload psr-4: PhpSaga\\Symfony\\ => src/
```

2 DependencyInjection/Configuration.php — Config tree

```
// Defines valid configuration structure
// Accessible via: $config = $this->processConfiguration(new Configuration(), $configs)

phpsaga: # Bundle config root
log_to_database: false # bool — persist audit log to DB
retry_attempts: 3 # int — RETRY strategy attempts
retry_delay_ms: 100 # int — initial retry delay
alert_channel: monolog.logger # string — Monolog service ID

// Use TreeBuilder with ->children()->booleanNode()/integerNode()/scalarNode()
// All nodes must have ->defaultValue() set
// All nodes must have ->info('description') for self-documenting config
```

3 DependencyInjection/PhpSagaExtension.php — Container wiring

```
class PhpSagaExtension extends AbstractExtension {
public function loadExtension(array $config, ContainerConfigurator $container,
ContainerBuilder $builder): void {
// 1. Load services from Resources/config/services.yaml
$container->import('../Resources/config/services.yaml');

// 2. Pass config values as container parameters
$container->parameters()
->set('phpsaga.log_to_database', $config['log_to_database'])
->set('phpsaga.retry_attempts', $config['retry_attempts'])
->set('phpsaga.retry_delay_ms', $config['retry_delay_ms']);

// 3. Auto-tag SagaStepInterface implementations
```

```
$builder->registerForAutoconfiguration(SagaStepInterface::class)
->addTag('phpsaga.step');
}
}
```

4 src/Attribute/Transactional.php — PHP 8 method attribute

```
<?php declare(strict_types=1);
namespace PhpSaga\Symfony\Attribute;

#[\Attribute(\Attribute::TARGET_METHOD)]
final class Transactional {
public function __construct(
// Which DB connection to use (matches Doctrine connection name)
public readonly string $connection = 'default',

// Force a brand new transaction even if one is already open
public readonly bool $newTransaction = false,
) {}
}

// Usage on a controller:
// #[Transactional]
// public function checkout(Request $request): Response { ... }

// Usage with explicit connection:
// #[Transactional(connection: 'tenant', newTransaction: true)]
```

5 src/Service/SagaRunner.php — Injectable service

```
class SagaRunner {
public function __construct(
private readonly EntityManagerInterface $em,
private readonly LoggerInterface $logger,
private readonly int $retryAttempts,
private readonly int $retryDelayMs,
) {}

public function run(array $steps): TransactionResult {
$this->em->beginTransaction();
try {
```

```
$result = Transaction::run($steps);
if ($result->wasSuccessful()) {
$this->em->flush();
$this->em->commit();
} else {
$this->em->rollback();
}
return $result;
} catch (\Throwable $e) {
$this->em->rollback();
$this->logger->critical('PhpSaga transaction failed', ['error' => $e->getMessage()]);
throw $e;
}
}
}
```

6 src/EventListener/TransactionalListener.php — Kernel event

```
// Listens on: kernel.controller
// Purpose: detect #[Transactional] on controller method, wrap automatically

public function onKernelController(ControllerEvent $event): void {
$controller = $event->getController();

// Extract method from controller callable
[$object, $method] = is_array($controller) ? $controller : [$controller, '__invoke'];

// Use PHP Reflection to check for #[Transactional]
$reflection = new \ReflectionMethod($object, $method);
$attrs = $reflection->getAttributes(Transactional::class);
if (empty($attrs)) return; // Not annotated — do nothing

// Replace controller with wrapped version
$attr = $attrs[0]->newInstance();
$event->setController(function() use ($object, $method, $attr, $event) {
return $this->sagaRunner->run( // wraps in EM transaction
$object->$method(...$event->getRequest()->attributes->all())
);
});
}
```

7 src/Contracts/SagaStepInterface.php + auto-discovery

```
interface SagaStepInterface {
// The main action — called by SagaRunner
public function execute(): mixed;

// Compensation — called on failure in reverse order
// MUST be idempotent
public function compensate(): void;

// Human-readable step identifier for audit log
public function getStepName(): string;

// One of Step::RETRY|LOG_AND_CONTINUE|ALERT_HUMAN|THROW
public function getCompensationFailureStrategy(): int;
}

// Services implementing this interface are auto-tagged 'phpsaga.step'
// and injectable anywhere via SagaStepInterface $myStep
```

8 tests/ — KernelTestCase + WebTestCase

```
Test 1: SagaRunner autowires from container
$runner = self::getContainer()->get(SagaRunner::class)
Assert instanceof SagaRunner ✓

Test 2: #[Transactional] on controller method
Create test controller with #[Transactional] method that throws
Make HTTP request to the route
Assert: EntityManager->rollback() called ✓
Assert: Exception propagated as 500 response ✓

Test 3: Doctrine EM rolls back on failure
SagaRunner->run() with a step that throws
Assert: EntityManager->rollback() called ✓
Assert: EntityManager->flush() NOT called ✓

Test 4: Bundle loads without error
Boot kernel with phpsaga bundle config
Assert: No ContainerBuilder exceptions ✓
Assert: All services resolve ✓
```

```
Test 5: SagaStepInterface auto-discovery
Create class implementing SagaStepInterface
Assert: Tagged as phpsaga.step in compiled container ✓
```

■ EXPECTED OUTCOMES — VERIFY BEFORE MARKING DONE

- ✓ Bundle registers in Symfony 6 and Symfony 7 without errors
- ✓ #[Transactional] annotation auto-wraps controller methods
- ✓ SagaRunner injectable via constructor injection in any service
- ✓ Doctrine EntityManager rolls back on any Throwable (not just Exception)
- ✓ SagaStepInterface implementations auto-tagged via registerForAutoconfiguration
- ✓ Configuration tree validates — typos in phpsaga.yaml produce clear errors
- ✓ All 5 KernelTestCase/WebTestCase tests pass on both Symfony 6 and 7
- ✓ Zero static calls, zero service locators in bundle code

v2.5

Persistent Saga State

Survive server crashes mid-compensation. Recovery engine. CLI recovery tool.

PHP 8.1+ · PDO · NDJSON · UUID v4 · CLI SAPI

■ **PREREQUISITE: PhpSaga v1.0 is complete. All 6 tests pass. v1.5 and v2.0 are complete.**

■ AGENT CONTEXT

ROLE: Systems engineer with deep knowledge of distributed systems,
event sourcing, idempotency, and crash recovery patterns.
GOAL: Add optional persistent saga state so transactions survive server
crashes mid-compensation. Build a recovery engine and CLI tool.
OUTPUT: Persistence layer (PDO + File drivers), SagaRecovery engine,
CLI recovery script. v1.0 behavior unchanged without persistence.
QUALITY: All persistence writes are append-only and idempotent.
Recovery is safe to run multiple times without side effects.

■ ABSOLUTE CONSTRAINTS

Persistence is OPTIONAL — v1.0 must behave identically when $persistence=null
ALL persistence writes MUST be append-only INSERT — no UPDATE or DELETE ever
ALL writes MUST be idempotent — safe to replay on crash recovery
Recovery MUST NEVER re-fire a compensation that already has COMPENSATION_COMPLETED
FilePersistence writes MUST be atomic — use temp file + rename() pattern
SagaId MUST be UUID v4 — never use sequential IDs
CLI tool MUST exit with code 0 on full success, code 1 on any partial failure
StateTracker interface MUST remain identical when no persistence provided

IMPLEMENTATION STEPS — Execute in strict order

1 src/SagaId.php — UUID v4 generator

```
final class SagaId {
public static function generate(): string {
// Generate UUID v4 without external dependencies
```

```
$data = random_bytes(16);
$data[6] = chr(ord($data[6]) & 0x0f | 0x40); // version 4
$data[8] = chr(ord($data[8]) & 0x3f | 0x80); // variant bits
return vsprintf('%s%s-%s-%s-%s-%s%s%s', str_split(bin2hex($data), 4));
}

public static function isValid(string $id): bool {
return (bool) preg_match(
'/^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$/i',
$id
);
}
}
```

2 src/Persistence/SagaPersistence.php — Interface

```
interface SagaPersistence {
// Call ONCE at transaction start — records step names for recovery
public function startSaga(string $sagaId, array $stepNames): void;

// Call after each step completes — $result is JSON-serializable
public function markStepCompleted(string $sagaId, string $step, mixed $result): void;

// Call when a step fails (only the failed step, not compensations)
public function markStepFailed(string $sagaId, string $step, string $error): void;

// Call after each compensation successfully fires
public function markCompensationCompleted(string $sagaId, string $step): void;

// Call when a compensation itself fails
public function markCompensationFailed(string $sagaId, string $step, string $err): void;

// Call when the entire transaction completes successfully
public function markSagaCompleted(string $sagaId): void;

// Call when compensation is fully done (all compensations fired)
public function markSagaRolledBack(string $sagaId): void;

// Returns saga_ids that started but never completed or rolled back
public function getIncompleteSagas(): array;

// Returns full event history for a saga (ordered by created_at ASC)
```

```
public function getSagaState(string $sagaId): array;
}
```

3 src/Persistence/Schema/saga_log.sql — Database schema

```
CREATE TABLE phpsaga_log (
id BIGINT AUTO_INCREMENT PRIMARY KEY,
saga_id VARCHAR(36) NOT NULL,
step_name VARCHAR(255) NOT NULL DEFAULT '',
event VARCHAR(50) NOT NULL,
result TEXT NULL, -- JSON encoded step result
error TEXT NULL, -- Error message on failure
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

INDEX idx_saga_id (saga_id),
INDEX idx_event (event),
INDEX idx_created (created_at)
);

-- NEVER add UPDATE or DELETE to this table
-- It is an append-only event log
-- All queries are: INSERT (write) or SELECT (read)
```

4 src/Persistence/PdoPersistence.php — Database driver

```
// ALL writes are INSERT — never UPDATE, never DELETE
// Concurrent safety: use SELECT ... FOR UPDATE when reading incomplete sagas

private function insert(string $sagaId, string $step, string $event,
mixed $result=null, ?string $error=null): void {
$stmt = $this->pdo->prepare(
'INSERT INTO phpsaga_log (saga_id, step_name, event, result, error)
VALUES (:saga_id, :step_name, :event, :result, :error)'
);
$stmt->execute([
':saga_id' => $sagaId,
':step_name' => $step,
':event' => $event,
':result' => $result !== null ? json_encode($result) : null,
':error' => $error,
]);
```

```
}

// getIncompleteSagas(): SELECT saga_id WHERE saga_id NOT IN
// (SELECT saga_id WHERE event IN ('SAGA_COMPLETED','SAGA_ROLLED_BACK'))
// getSagaState(): SELECT * WHERE saga_id = ? ORDER BY created_at ASC
```

5 src/Persistence/FilePersistence.php — Filesystem driver

```
// Each saga = one NDJSON file: {storageDir}/{sagaId}.json
// NDJSON = newline-delimited JSON (one event object per line)
// Atomic write pattern: write to tmp file → rename() into place

private function appendEvent(string $sagaId, array $event): void {
$file = $this->storageDir . '/' . $sagaId . '.json';
$tmpFile = $file . '.tmp.' . getmypid();

// Read existing content
$existing = file_exists($file) ? file_get_contents($file) : '';

// Append new event line
$line = json_encode($event) . PHP_EOL;

// Write to temp file, then atomically rename
file_put_contents($tmpFile, $existing . $line);
rename($tmpFile, $file); // atomic on POSIX systems
}

// getIncompleteSagas(): scan dir for *.json files, parse events,
// return sagaIds without SAGA_COMPLETED or SAGA_ROLLED_BACK event
```

6 Update src/Engine/StateTracker.php — Add persistence

```
// Add optional persistence to constructor (default null = v1.0 behavior)
__construct(private readonly ?SagaPersistence $persistence = null)

// Update markCompleted to also persist:
public function markCompleted(string $name, mixed $result = null): void {
$this->state[$name] = ['status' => 'completed', 'result' => $result];
$this->persistence?->markStepCompleted($this->sagaId, $name, $result);
}
```

```
// Add sagaId tracking:
public function setSagaId(string $sagaId): void { $this->sagaId = $sagaId; }

// Add state restoration from persistence (for recovery):
public function loadFromPersistence(string $sagaId): void {
$events = $this->persistence->getSagaState($sagaId);
foreach ($events as $event) {
if ($event['event'] === 'STEP_COMPLETED') {
$this->state[$event['step_name']] = ['status' => 'completed', ...];
}
// Replay STEP_FAILED, COMPENSATION_COMPLETED etc.
}
}
```

7 src/Recovery/SagaRecovery.php — Crash recovery engine

```
// ALGORITHM — implement exactly:

public function recover(): array { // returns RecoveryResult[]
$incompleteSagas = $this->persistence->getIncompleteSagas();
$results = [];

foreach ($incompleteSagas as $sagaId) {
$events = $this->persistence->getSagaState($sagaId);

// 1. Find which steps COMPLETED
$completedSteps = array_filter($events,
fn($e) => $e['event'] === 'STEP_COMPLETED');

// 2. Find which compensations ALREADY FIRED successfully
$compensatedSteps = array_filter($events,
fn($e) => $e['event'] === 'COMPENSATION_COMPLETED');

// 3. Steps needing compensation = completed - already compensated
$needsCompensation = array_diff(
array_column($completedSteps, 'step_name'),
array_column($compensatedSteps, 'step_name')
);

// 4. Run compensation for ONLY uncompensated steps (reverse order)
$fired = [];
foreach (array_reverse($needsCompensation) as $stepName) {
```

```
// find step in $this->stepRegistry, call compensation
// record result, add to $fired
}

$results[] = new RecoveryResult($sagaId, $fired, ...);
}
return $results;
}
```

8 bin/phpsaga-recover — Standalone CLI tool

```
#!/usr/bin/env php
<?php
// Standalone PHP CLI — no framework required
// Config loaded from: phpsaga.php in project root OR .env file

// Expected output format:
// PhpSaga Recovery Tool
// ====================
// Found 2 incomplete saga(s).
//
// [1/2] Saga: 550e8400-e29b-41d4-a716-446655440000
// Resumed from: send_sms
// Compensations fired: [store_otp, send_sms]
// Status: FULLY RECOVERED
//
// [2/2] Saga: 6ba7b810-9dad-11d1-80b4-00c04fd430c8
// Resumed from: send_email
// Compensations fired: [send_email]
// Status: PARTIAL FAILURE — 1 compensation error
//
// Recovery complete. 1 full, 1 partial.

exit($hasFailures ? 1 : 0);
```

9 tests/ — Persistence + Recovery tests

```
Test 1: Crash simulation
Run Transaction::run() with PdoPersistence, simulate crash after step 2
Create fresh Transaction instance (simulates restart)
Run SagaRecovery::recover()
```

```
Assert: incomplete saga found ✓
Assert: only step 1 compensated (step 2 never completed) ✓

Test 2: Idempotency — no double compensation
COMPENSATION_COMPLETED exists for step 1 in DB
Run recovery again
Assert: step 1 compensation NOT called again ✓

Test 3: FilePersistence atomic writes
Write 2 events concurrently from different processes
Assert: both events present in file ✓
Assert: no corrupted JSON lines ✓

Test 4: v1.0 backward compatibility
Transaction::run($steps) with no persistence param
Assert: behaves identically to v1.0 ✓
Assert: no persistence calls made ✓

Test 5: CLI exit codes
Recovery with all success → exit code 0 ✓
Recovery with partial failure → exit code 1 ✓
```

■ EXPECTED OUTCOMES — VERIFY BEFORE MARKING DONE

- ✓ Transaction::run() with no persistence param = identical to v1.0 behavior
- ✓ Crash simulation test: recovery finds incomplete sagas correctly
- ✓ Idempotency test: already-compensated steps never re-compensated
- ✓ PdoPersistence: zero UPDATE or DELETE statements in all SQL queries
- ✓ FilePersistence: atomic writes via temp-file + rename — no corruption
- ✓ SagaRecovery resumes compensation from exact crash point
- ✓ CLI: exit code 0 on full recovery, exit code 1 on any failure
- ✓ All 5 tests pass including concurrent write safety test

v3.0

Node.js / TypeScript Port

Clean TypeScript reimplementation. Full async/await. Knex + Prisma support.

TypeScript 5+ · Node 20+ · ESM · Vitest · npm

■ **PREREQUISITE: PhpSaga v1.0 is complete and passing. Use it as the reference implementation.**

■ AGENT CONTEXT

ROLE: Senior TypeScript engineer. Expert in async patterns, generics,

ESM modules, and npm package publishing.

GOAL: Reimplement PhpSaga in TypeScript for Node.js. NOT a PHP wrapper.

This is a clean, idiomatic TypeScript library with the same semantics.

OUTPUT: npm-publishable @phpsaga/node package. ESM + CJS dual output.

Full TypeScript generics. Vitest test suite. Express middleware.

QUALITY: Strict TypeScript. Zero 'any' types. Never throws — always returns result.

■ ABSOLUTE CONSTRAINTS

Zero 'any' types — use 'unknown' with type guards where necessary

Transaction.run() MUST NEVER throw — errors always in TransactionResult

Steps MUST execute SEQUENTIALLY — async parallelism breaks compensation

Compensation MUST run in REVERSE order — same as PHP implementation

Package MUST support ESM (primary) and CommonJS (fallback) via dual exports

All tests MUST be async — use async/await in every test case

TypeScript declaration files (.d.ts) MUST be generated in dist/

Step generic MUST be inferred without explicit type annotation by callers

IMPLEMENTATION STEPS — Execute in strict order

1 package.json + tsconfig.json — Project setup

```
// package.json
{
"name": "@phpsaga/node",
```

```
"version": "3.0.0",
"type": "module",
"exports": {
".": {
"import": "./dist/index.js",
"require": "./dist/index.cjs"
}
},
"types": "./dist/index.d.ts",
"scripts": { "build": "tsc", "test": "vitest run", "typecheck": "tsc --noEmit" },
"devDependencies": { "typescript": "^5.0", "vitest": "^1.0", "@types/node": "^20" }
}

// tsconfig.json
{ "compilerOptions": {
"target": "ES2022", "module": "NodeNext", "moduleResolution": "NodeNext",
"strict": true, "noImplicitAny": true, "exactOptionalPropertyTypes": true,
"declaration": true, "declarationMap": true, "outDir": "dist"
} }
```

2 src/types.ts — All TypeScript interfaces

```
export interface TransactionResult {
readonly success: boolean;
readonly auditLog: readonly AuditEntry[];
readonly failedStep?: string;
readonly error?: Error;
}

export interface AuditEntry {
readonly event: AuditEvent;
readonly stepName?: string;
readonly timestamp: Date;
readonly context?: Readonly<Record<string, unknown>>;
}

export type AuditEvent =
| 'STEP_STARTED' | 'STEP_COMPLETED' | 'STEP_FAILED'
| 'COMPENSATION_STARTED' | 'COMPENSATION_COMPLETED' | 'COMPENSATION_FAILED'
| 'TRANSACTION_COMPLETED' | 'TRANSACTION_ROLLED_BACK';

// ORM transaction types (duck-typed — no dependency on Knex or Prisma)
```

```
export interface KnexLike { commit(): Promise<void>; rollback(): Promise<void>; }
export interface PrismaLike { $commit(): Promise<void>; $rollback(): Promise<void>; }
export type DbTransaction = KnexLike | PrismaLike;
```

3 src/failure/RollbackFailure.ts + src/Step.ts

```
// RollbackFailure.ts
export enum RollbackFailureStrategy {
RETRY = 'RETRY',
LOG_AND_CONTINUE = 'LOG_AND_CONTINUE',
ALERT_HUMAN = 'ALERT_HUMAN',
THROW = 'THROW',
}

// Step.ts — fully generic
export class Step<T = unknown> {
private constructor(
readonly name: string,
readonly action: () => Promise<T> | T,
readonly compensation?: () => Promise<void> | void,
readonly failStrategy: RollbackFailureStrategy =
RollbackFailureStrategy.LOG_AND_CONTINUE,
) {}

static make<T>(action: () => Promise<T> | T, name = ''): Step<T>
rollback(fn: () => Promise<void> | void): this
onRollbackFailure(s: RollbackFailureStrategy): this
}
```

4 src/engine/StateTracker.ts + AuditLogger.ts

```
// StateTracker.ts
type StepState = {
status: 'pending' | 'completed' | 'failed';
result?: unknown;
error?: Error;
};

export class StateTracker {
private state = new Map<string, StepState>();
private completionOrder: string[] = []; // tracks ORDER of completion
```

```
markCompleted(name: string, result: unknown): void
markFailed(name: string, error: Error): void
getCompletedSteps(): string[] // returns in COMPLETION ORDER
wasCompleted(name: string): boolean
}

// AuditLogger.ts
export class AuditLogger {
private readonly entries: AuditEntry[] = [];
// Optional custom logger injection:
constructor(private readonly logger?: (entry: AuditEntry) => void) {}
log(event: AuditEvent, stepName?: string, context?: Record<string,unknown>): void
getLogs(): readonly AuditEntry[]
}
```

5 src/engine/ExecutionEngine.ts — Async sequential runner

```
export class ExecutionEngine {
constructor(
private readonly tracker: StateTracker,
private readonly logger: AuditLogger,
) {}

async execute(steps: Step[]): Promise<void> {
for (const step of steps) { // SEQUENTIAL — NOT Promise.all()
this.logger.log('STEP_STARTED', step.name);
try {
const result = await step.action(); // await even sync functions
this.tracker.markCompleted(step.name, result);
this.logger.log('STEP_COMPLETED', step.name);
} catch (e) {
const error = e instanceof Error ? e : new Error(String(e));
this.tracker.markFailed(step.name, error);
this.logger.log('STEP_FAILED', step.name, { error: error.message });
throw error; // stop execution, trigger compensation
}
}
}
}
```

6 src/engine/CompensationEngine.ts — Reverse async rollback

```
export class CompensationEngine {
async compensate(steps: Step[], completedNames: string[]): Promise<void> {
// completedNames is already in completion order
// We must reverse it for compensation
const toCompensate = [...completedNames].reverse();

for (const name of toCompensate) {
const step = steps.find(s => s.name === name);
if (!step?.compensation) continue; // no compensation = DB op, skip

this.logger.log('COMPENSATION_STARTED', name);
try {
await step.compensation();
this.logger.log('COMPENSATION_COMPLETED', name);
} catch (e) {
const error = e instanceof Error ? e : new Error(String(e));
this.logger.log('COMPENSATION_FAILED', name, { error: error.message });
await this.handler.handle(step, error);
}
}
}
}
```

7 src/Transaction.ts — Main entry point

```
export class Transaction {
static async run(
steps: Step[],
db?: DbTransaction,
): Promise<TransactionResult> { // NEVER throws

const tracker = new StateTracker();
const logger = new AuditLogger();
const engine = new ExecutionEngine(tracker, logger);
const compensator = new CompensationEngine(tracker, logger, handler);

try {
await engine.execute(steps);
await commitDb(db); // duck-type: Knex or Prisma
```

```
logger.log('TRANSACTION_COMPLETED');
return { success: true, auditLog: logger.getLogs() };

} catch (error) {
await compensator.compensate(steps, tracker.getCompletedSteps());
await rollbackDb(db);
logger.log('TRANSACTION_ROLLED_BACK');
return {
success: false,
auditLog: logger.getLogs(),
failedStep: tracker.getFailedStep(),
error: error instanceof Error ? error : new Error(String(error)),
};
}
}
}
```

8 src/middleware/express.ts + src/index.ts

```
// express.ts
import type { Request, Response, NextFunction, RequestHandler } from 'express';

export function transactional(
getSteps: (req: Request) => Step[],
): RequestHandler {
return async (req: Request, res: Response, next: NextFunction) => {
const result = await Transaction.run(getSteps(req));
if (result.success) {
next();
} else {
next(result.error);
}
};
}

// index.ts — public exports only (tree-shakeable)
export { Transaction } from './Transaction.js';
export { Step } from './Step.js';
export { RollbackFailureStrategy } from './failure/RollbackFailure.js';
export { transactional } from './middleware/express.js';
export type { TransactionResult, AuditEntry, DbTransaction } from './types.js';
```

9 tests/ — Vitest test suite (all async)

```
Test 1: Sequential execution
Steps record timestamps. Assert step2.start > step1.end (sequential, not parallel) ✓

Test 2: Compensation reverse order
4 steps complete. 5th step fails.
Compensation call log = ['step4','step3','step2','step1'] ✓

Test 3: TransactionResult never throws
step that throws Error → Transaction.run() returns result, does not throw ✓
result.success === false ✓ | result.error instanceof Error ✓

Test 4: Knex mock integration
Create mock with commit/rollback methods
Success path → mock.commit() called once ✓
Failure path → mock.rollback() called once ✓

Test 5: Express middleware
Mount transactional() middleware
Failed step → next(error) called with Error instance ✓
Success → next() called with no arguments ✓

Test 6: TypeScript generics
Step.make(async () => 42) infers as Step<number> without annotation ✓
Step.make(async () => 'hello') infers as Step<string> ✓
```

■ EXPECTED OUTCOMES — VERIFY BEFORE MARKING DONE

- ✓ npm install @phpsaga/node resolves without errors
- ✓ TypeScript declaration files generated correctly in dist/
- ✓ Transaction.run() never throws — all errors in result object
- ✓ Sequential execution verified: step timestamps prove non-parallel execution
- ✓ Compensation exact reverse order verified by call log in Test 2
- ✓ Knex mock: commit on success, rollback on failure
- ✓ Express middleware: next(error) on failure, next() on success
- ✓ Step generic inferred without explicit annotation (Test 6)
- ✓ Both ESM import and CommonJS require work correctly

✓ tsc --noEmit passes with zero TypeScript errors

v4.0

Python Port

Sync + async transparent execution. Django + FastAPI integrations.

Python 3.11+ · asyncio · Django 4/5 · FastAPI · pytest-asyncio

■ PREREQUISITE: PhpSaga v1.0 is complete and passing. Use it as the reference implementation.

■ AGENT CONTEXT

ROLE: Senior Python engineer. Expert in asyncio, type hints, dataclasses,

Django ORM, SQLAlchemy async, and FastAPI middleware.

GOAL: Port PhpSaga to Python. Support sync and async callables transparently.

Django and FastAPI developers get native-feeling integrations.

OUTPUT: pip-installable 'phpsaga' package. Sync + async entry points.

Django @transactional decorator. FastAPI middleware. All tests pass.

QUALITY: Type hints everywhere. mypy --strict passes. Python 3.11+ features.

■ ABSOLUTE CONSTRAINTS

Type hints on ALL functions and methods — no bare variables without types

mypy --strict MUST pass with zero errors

Sync callables and async callables MUST be transparently mixed in same step list

Transaction.run() (sync) MUST NOT use asyncio.run() internally — use loop detection

Transaction.run_async() MUST be a true coroutine — awaitable with 'await'

Django @transactional MUST integrate with django.db.transaction.atomic()

FastAPI middleware MUST work with both sync and async route handlers

All dataclasses MUST use frozen=True where fields shouldn't change after creation

pytest --asyncio-mode=auto MUST run all tests including async ones correctly

IMPLEMENTATION STEPS — Execute in strict order

1 pyproject.toml — Package configuration

```
[build-system]
requires = ["hatchling"]
```

```
build-backend = "hatchling.build"

[project]
name = "phpsaga"
version = "4.0.0"
requires-python = ">=3.11"
dependencies = [] # zero runtime dependencies

[project.optional-dependencies]
django = ["django>=4.0"]
fastapi = ["fastapi>=0.100", "starlette>=0.27"]
dev = ["pytest>=7", "pytest-asyncio>=0.21", "mypy>=1.0", "httpx>=0.24"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
```

2 phpsaga/failure/strategies.py — Enums and types

```
from enum import Enum
from typing import Callable, Any, Awaitable, Union

class RollbackStrategy(Enum):
RETRY = "retry"
LOG_AND_CONTINUE = "log_and_continue"
ALERT_HUMAN = "alert_human"
RAISE = "raise"

# Type alias for callables that can be sync or async
SyncCallable = Callable[[], Any]
AsyncCallable = Callable[[], Awaitable[Any]]
AnyCallable = Union[SyncCallable, AsyncCallable]
```

3 phpsaga/step.py — Fluent step builder

```
from dataclasses import dataclass, field
from typing import Self
from .failure.strategies import RollbackStrategy, AnyCallable

@dataclass
class Step:
name: str
```

```
action: AnyCallable
compensation: AnyCallable | None = field(default=None, repr=False)
strategy: RollbackStrategy = RollbackStrategy.LOG_AND_CONTINUE

@classmethod
def make(cls, action: AnyCallable, name: str = '') -> 'Step':
return cls(name=name or action.__name__, action=action)

def rollback(self, fn: AnyCallable) -> Self:
# Returns new instance (immutable pattern)
from dataclasses import replace
return replace(self, compensation=fn)

def on_rollback_failure(self, strategy: RollbackStrategy) -> Self:
from dataclasses import replace
return replace(self, strategy=strategy)
```

4 phpsaga/engine/state_tracker.py — Step state tracking

```
from dataclasses import dataclass, field
from typing import Any, Literal

@dataclass
class StepState:
name: str
status: Literal['pending', 'completed', 'failed']
result: Any = None
error: Exception | None = None

class StateTracker:
def __init__(self) -> None:
self._state: dict[str, StepState] = {}
self._order: list[str] = [] # completion order

def mark_completed(self, name: str, result: Any = None) -> None:
self._state[name] = StepState(name, 'completed', result)
self._order.append(name)

def mark_failed(self, name: str, error: Exception) -> None:
self._state[name] = StepState(name, 'failed', error=error)

def get_completed_steps(self) -> list[str]:
```

```
return list(self._order) # in completion order

def was_completed(self, name: str) -> bool:
return self._state.get(name, StepState('', 'pending')).status == 'completed'
```

5 phpsaga/engine/execution.py — Transparent sync/async runner

```
import asyncio, inspect
from typing import Any
from ..step import Step

# Core helper: run any callable, sync or async
async def _call(fn: Any) -> Any:
if inspect.iscoroutinefunction(fn):
return await fn()
return fn() # sync callable called directly in async context

class ExecutionEngine:
async def execute(self, steps: list[Step]) -> None:
for step in steps: # SEQUENTIAL — not asyncio.gather()
try:
result = await _call(step.action)
self.tracker.mark_completed(step.name, result)
self.logger.log('STEP_COMPLETED', step.name)
except Exception as e:
self.tracker.mark_failed(step.name, e)
self.logger.log('STEP_FAILED', step.name, {'error': str(e)})
raise # stop execution
```

6 phpsaga/transaction.py — Dual sync/async entry points

```
from dataclasses import dataclass
from typing import Any

@dataclass(frozen=True)
class TransactionResult:
success: bool
audit_log: tuple # immutable — frozen dataclass
failed_step: str | None = None
error: Exception | None = None
```

```
class Transaction:
@staticmethod
def run(steps: list, db: Any = None) -> TransactionResult:
# Sync entry point — runs async engine in a new event loop
# ONLY if not already in an event loop (avoids RuntimeError)
try:
loop = asyncio.get_running_loop()
# Already in async context — raise helpful error
raise RuntimeError('Use Transaction.run_async() in async context')
except RuntimeError:
return asyncio.run(Transaction._run(steps, db))

@staticmethod
async def run_async(steps: list, session: Any = None) -> TransactionResult:
return await Transaction._run(steps, session)

@staticmethod
async def _run(steps: list, db: Any = None) -> TransactionResult:
# Core implementation — used by both sync and async
...
```

7 phpsaga/django.py — Django integration

```
from functools import wraps
from django.db import transaction as django_transaction
from .transaction import Transaction

# Decorator for function-based views
def transactional(view_func):
@wraps(view_func)
def wrapper(request, *args, **kwargs):
with django_transaction.atomic():
return view_func(request, *args, **kwargs)
return wrapper

# Mixin for class-based views
class TransactionalMixin:
def get_saga_steps(self) -> list:
# Override in subclass to return list of Step objects
return []

def dispatch(self, request, *args, **kwargs):
```

```
result = Transaction.run(self.get_saga_steps())
if not result.success:
raise result.error
return super().dispatch(request, *args, **kwargs)
```

8 phpsaga/fastapi.py — FastAPI middleware

```
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
from starlette.responses import Response
from .transaction import Transaction

class TransactionalMiddleware(BaseHTTPMiddleware):
async def dispatch(self, request: Request,
call_next) -> Response:
try:
response = await call_next(request)
return response
except Exception as e:
# PhpSaga compensation already fired inside the route handler
# This middleware handles DB session rollback
raise

# Usage in main.py:
# app = FastAPI()
# app.add_middleware(TransactionalMiddleware)

# Inside a route — use Transaction.run_async():
# result = await Transaction.run_async([
# Step.make(async_action).rollback(async_compensate),
# ], session=db)
```

9 tests/ — pytest + pytest-asyncio test suite

```
Test 1: Sync happy path
Transaction.run([step1, step2, step3])
result.success is True ✓ | len(result.audit_log) > 0 ✓

Test 2: Async happy path
result = await Transaction.run_async([async_step1, async_step2])
result.success is True ✓
```

```
Test 3: Mixed sync + async steps
steps = [sync_step, async_step, sync_step2]
All three execute correctly ✓ | result.success is True ✓

Test 4: Compensation order
4 steps. Step 3 fails.
Compensation call order = ['step2', 'step1'] (reverse of completed) ✓

Test 5: Django @transactional decorator
Mock Django ORM. Decorated view raises exception.
Assert: transaction.rollback() called ✓

Test 6: FastAPI middleware
Create test app with TransactionalMiddleware
Use httpx AsyncClient to make request
Step failure → 500 response ✓ | DB session rolled back ✓

Run: pytest --asyncio-mode=auto (all tests must pass)
Run: mypy phpsaga/ --strict (zero errors)
```

■ EXPECTED OUTCOMES — VERIFY BEFORE MARKING DONE

- ✓ pip install phpsaga installs without errors
- ✓ from phpsaga import Transaction, Step, RollbackStrategy works
- ✓ Sync and async callables mixed transparently — Test 3 passes
- ✓ Transaction.run() raises RuntimeError if called from async context (not silently broken)
- ✓ await Transaction.run_async() works correctly from async context
- ✓ Compensation runs in exact reverse order — Test 4 passes
- ✓ Django @transactional rolls back on exception — Test 5 passes
- ✓ FastAPI middleware test passes with httpx AsyncClient — Test 6 passes
- ✓ pytest --asyncio-mode=auto: all 6 tests pass
- ✓ mypy phpsaga/ --strict: zero errors

All 6 versions. All steps. All outcomes.

Build it. Test it. Ship it.